BreaQ 2026 Blue Track Challenge

End-to-End BB84 QKD Implementation & Threat Modeling

Final Architecture Report & Post-Mortem

October 24, 2026

Abstract

This report outlines our architectural strategy and mathematical implementation of the BB84 Quantum Key Distribution (QKD) protocol. The objective was to transform a simulated, noisy quantum transmission into a provably secure cryptographic key over HTTP without the use of a quantum SDK. We detail our shift to a Stateful Micro-Endpoint Architecture, the resolution of critical index-desynchronization, 8-bit integer overflow mitigation, our 4-pass Cascade retry loop, and a zero-memory virtual Toeplitz matrix implementation for Privacy Amplification.

1 Architecture: Stateful Micro-Endpoints

A common failure mode in QKD software is entangling cryptographic mathematics with HTTP network routing. To ensure stability during the recursive network queries required by the Cascade algorithm, we implemented a **Stateful Micro-Endpoint Architecture**.

The simulated "Alice" (transmitter) was built using FastAPI. To model a physical hardware baseline, Alice utilized an in-memory session dictionary, securely isolating her original arrays of $200,000$ bases and digital bits. The client "Bob" was built independently using Python's `requests` module, cleanly isolating the statistical analysis and binary search logic from the server's data persistence layer. Both systems strictly enforced `np.int8` data types to ensure consistent byte-packing during the final SHA-256 verification.

2 Sifting Phase and Index Synchronization

During early iterations, our Hoeffding bound calculation returned a catastrophic raw error rate ($\hat{Q}$) of nearly 50%. The root cause was classical index desynchronization: when Bob discarded his mismatched bases during Sifting, his array shrank. Querying Alice using these new, relative indices compared completely unrelated photons.

Resolution: We implemented an absolute-tracking vector (`raw_indices`) that mapped Bob's local sifted key coordinates back to the absolute physical photon indices originally emitted by Alice. Passing these raw indices over HTTP restored mathematical synchronization, yielding an accurate physical QBER of approximately 2.1%.

3 Cascade Error Correction and Edge Cases

With the secure line established, we implemented a 4-pass Cascade algorithm, dynamically partitioning the key into blocks of length $b \approx \max(4, 0.73/\hat{Q})$. To ensure complete error eradication, we addressed three critical computational hurdles:

1. **Tail-End Truncation:** Standard floor division (`//`) left the final fractional block of the key unchecked, allowing errors to survive. We shifted to a ceiling function (`math.ceil`) and dynamically capped the slicing bounds (`min((i + 1) * b, len(secure_key))`), ensuring 100% array coverage without triggering `IndexError` exceptions.

2. **8-Bit Parity Overflows:** Calculating parity via `np.sum(block) % 2` on large blocks resulted in 8-bit integer overflow, corrupting the parity checks. We replaced this with `np.bitwise_xor.reduce`, mathematically guaranteeing safe parity evaluation regardless of block size.

3. **The Retry Loop:** To account for dense error clusters that survive the permutations in passes 2-4 (without implementing complex backtracking), we wrapped the Cascade logic in a 3-attempt retry loop per the protocol specification, successfully yielding identical SHA-256 verification hashes.

4 Privacy Amplification: The 29GB RAM Overflow

Privacy Amplification requires mathematically erasing the information leaked to Eve during the Cascade parity checks ($leak_{EC}$). The target secure key length (l) was calculated via the Shannon entropy of the Hoeffding bound (Q_u):

$$l = n - n \cdot h(Q_u) - leak_{EC} - 2\log_2\left(\frac{1}{\epsilon_{PA}}\right) \tag{1}$$

For a standard transmission, compressing an $85,000$-bit key down to $45,000$ bits required a Toeplitz matrix of size $45,000 \times 85,000$. Calling `scipy.linalg.toeplitz` coupled with `np.dot()` attempted to allocate 29 GB of standard RAM, crashing the system.

Memory-Safe Optimization: Because a Toeplitz matrix consists of shifted diagonal values, we utilized NumPy's `sliding_window_view`. This created a 0-byte "virtual" 2D matrix. We then processed the GF(2) matrix multiplication in chunks of 2,000 rows using bitwise AND (`&`) and XOR-reduce operations. This optimization dropped memory consumption to negligible levels and completed the geometric compression in under 1.5 seconds, yielding a mathematically secure cryptographic key.

Appendix: AI Collaboration Log

In accordance with the hackathon transparency guidelines, Generative AI was utilized as a pair-programming tool. Our strategy was to maintain architectural control and define the mathematical hypotheses, while utilizing the AI to generate boilerplate scaffolding, explain dense mathematical proofs, and suggest library-specific memory optimizations.

Below is a representative log of prompts submitted during our engineering sessions:

Prompt 1: Scaffolding the Stateful Server
"I need to set up a robust, stateful FastAPI server for 'Alice' to handle BB84 quantum key distribution requests. Can you provide a solid template that manages concurrent sessions and tracks quantum states via a session ID without leaking memory?"

Prompt 2: Cryptographic Theory and Threat Modeling
"Before we implement step 10, can you explain the exact algorithmic approach behind the permutation Alice performs? I need to understand the theoretical proof of how exchanging this specific mapping over a public HTTP channel doesn't compromise the underlying bit values to Eve."

Prompt 3: Hypothesizing the Tail-End Array Bug
"I'm basically stuck on this side of the code. Every time I go down to the end of the Cascade algorithm (after 3 or 4 passes), a few hidden errors remain and my SHA-256 hashes mismatch. I suspect this might be an edge case due to the way we divide the blocks—specifically a remainder truncation issue caused by floor division. How can we implement a ceiling function safely to ensure 100% array coverage?"

Prompt 4: Resolving the Big Data Hardware Limit
"I've successfully cleared the Cascade errors, but I just hit a massive hardware limit during Privacy Amplification. How could I solve my Toeplitz matrix error? I'm getting the following traceback: 'numpy._core._exceptions._ArrayMemoryError: Unable to allocate 29.0 GiB for an array with shape (45588, 85401)'. Is there a memory-safe virtual matrix approach in SciPy or NumPy we can use to bypass standard RAM allocation?"